

Week16_예습과제_백재은

Ch.09 추천 시스템

01. 추천 시스템의 개요와 배경

a. 추천 시스템의 개요

전자 상거래 업체, 영화음악 콘텐츠 제공 업체 → 경쟁 과열 → 사용자 관심 지속 목적 추천 시스템 고도화에 투자

b. 온라인 스토어의 필요 요소

사용자가 무엇을 원하는지 빠르게 찾아내 사용자의 온라인 쇼핑을 즐겁게 함, 필수 구성 요소가 된 추천 시스템, 사용자의 선택 부담을 해결

많은 양의 고객과 상품 관련 데이터를 가지고 있으며 이를 통해 사용자가 흥미를 가질 만한 상품을 즉각적으로 추천하는 데 사용



c. 추천 시스템의 유형

콘텐츠 기반 필터링, 협업 필터링, 최근접 이웃 협업 필터링, 잠재 요인 협업 필터링

초창기 → 콘텐츠 기반 필터링, 최근접 이웃 기반 협업 필터링

넷플릭스 추천 시스템 경연 대회 → 행렬 분해 기법을 이용한 잠재 요인 협업 필터링 방식 우승, 해당 시스템 적용 기업

개인화 특성 강화 → 하이브리드 형식, 콘텐츠 기반 + 협업 기반 적절히 결합하여 사용

02. 콘텐츠 기반 필터링 추천 시스템

a. Intro

콘텐츠 기반 필터링 방식: 사용자가 특정 아이템을 매우 선호하는 경우, 비슷한 콘텐츠를 가진 다른 아이템을 추천하는 방식

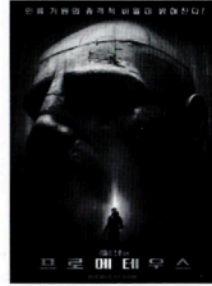
Ex. SF, 미스터리 장르 영화 컨택트에 8/10 - SF, 액션, 스릴러 장르 프로메테우스에 9/10 → 영화와 콘텐츠 감안해 적절하게 매칭되는 영화 추천



장르: SF, 드라마, 미스터리
감독: 드니 빌뇌브
출연: 에이미 아담스, 제레미 러너
키워드: 외계인 침공, 예술성, 스릴러 요소



장르: SF, 액션, 스릴러
감독: 드니 빌뇌브
출연: 라이언 고슬링, 해리슨 포드
키워드: 리들리 스콧 감독의 전작을 리메이크



장르: SF, 액션, 스릴러
감독: 리들리 스콧
출연: 노미 라마스, 마이클 패스벤더
키워드: 에일리언 프리퀄, 액션과 스릴러의 조화



사용자 선호 프로필

선호 장르: SF, 액션, 스릴러
선호 배우: 에이미 아담스, 마이클 패스벤더 등
선호 감독: 리들리 스콧, 드니 빌뇌브

03. 최근접 이웃 협업 필터링

협업 필터링 방식: 사용자가 아이템에 매긴 평점 정보나 상품 구매 이력과 같은 사용자 행동 양식만을 기반으로 추천을 수행하는 것

주요 목표: 사용자-아이템 평점 매트릭스와 같은 축적된 사용자 행동 데이터를 기반으로 사용자가 아직 평가하지 않은 아이템을 예측 평가하는 것 / 사용자가 평가한 다른 아이템을 기반으로 사용자가 평가하지 않은 아이템의 예측 평가를 도출하는 방식

추천 시스템 1 / 최근 이웃 방식

추천 시스템 2 / 잠재 요인 방식

→ 사용자 - 아이템 평점 행렬 데이터에만 의지해 추천 수행, 행: 개별 사용자, 열: 개별 아이템 / 사용자 아이디 행, 아이템 아이디 열 위치에 해당하는 값이 평점을 나타내는 형태

데이터가 레코드 레벨 형태인 사용자 - 아이템 평점 데이터라면 사용자 - 아이템 평점 행렬 형태로 변경

사용자 - 아이템 평점 행렬 형태 → 많은 아이템을 열로 가지는 다차원 행렬, 희소 행렬 특성

최근 이웃 방식 협업 필터링: 메모리 협업 필터링, 사용자 기반과 아이템 기반으로 다시 분류

사용자 기반: 당신과 비슷한 고객이 다음 상품도 구매함

→ 특정 사용자와 유사한 다른 사용자 Top-N 으로 선정, 해당 사용자가 좋아하는 아이템 추천, 유사도 측정 및 사용자 추출, 추천

아이템 기반: 이 상품을 선택한 다른 고객들은 다음 상품도 구매

→ 아이템이 가지는 속성으로는 상관없이 사용자들의 아이템에 대한 호불호 평가 척도가 유사한 아이템을 추천하는 기준이 되는 알고리즘, 사용자 기반 최근접 이웃 데이터 세트와 행렬이 서로 반대

사용자 기반보다 아이템 기반 협업 필터링이 정확도가 더 높음

유사도 측정을 위해 주로 코사인 유사도 이용

04. 잠재 요인 협업 필터링

a. 잠재 요인 협업 필터링의 이해

잠재 요인 협업 필터링: 사용자 - 아이템 평점 매트릭스 속에 숨어 있는 잠재 요인을 추출해 추천 예측을 할 수 있게 하는 기법

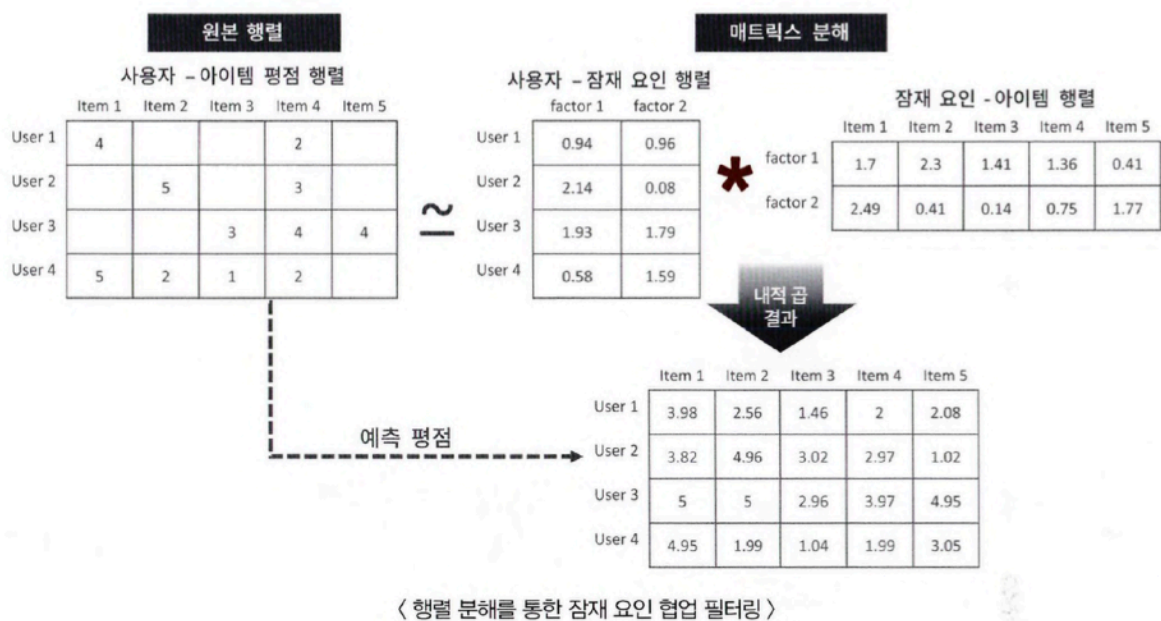
대규모 다차원 행렬을 차원 감소 기법으로 분해하는 과정에서 잠재 요인 추출, 행렬 분해

행렬 분해 기반의 잠재 요인 협업 필터링 - 넷플릭스 경연 대회에서 사용되면서 유명

사용자-아이템 평점 행렬 데이터만을 이용해 잠재 요인 끄집어 내는 것

잠재 요인 명확히 정의 불가, 잠재 요인을 기반으로 다차원 희소 행렬인 사용자-아이템 행렬 데이터를 저차원 밀집 행렬의 사용자-잠재 요인 행렬과 아이템 - 잠재 요인 행렬의 전치 행렬로 분해

해당 분해 두 행렬의 내적을 통해 새로운 예측 사용자-아이템 평점 행렬 데이터 생성, 사용자가 아직 평점을 부여하지 않는 아이템에 대한 예측 평점을 생성하는 것 = 잠재 요인 협업 필터링 알고리즘의 골자



사용자-아이템 평점 행렬 $R(u, i)$

- $R(u, i)$ 는 사용자 u 가 아이템 i 에 부여한 평점
- 예시:

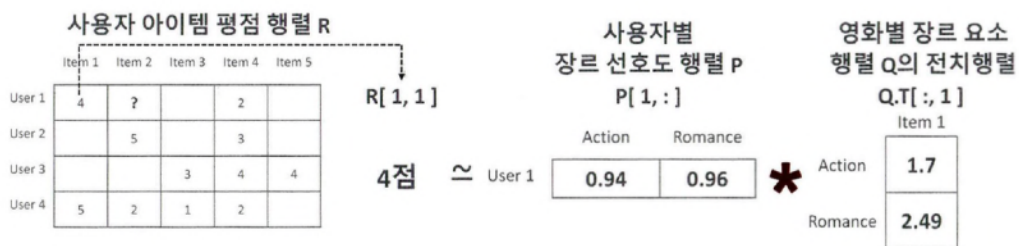
- $R(1, 1) = 4$
- $R(1, 4) = 2$

사용자-잠재요인 행렬 $P(u, k)$

- 사용자 u 의 영화 장르별 선호도를 잠재요인(latent factor)으로 표현한 행렬
- factor 설정:
 - factor 1: Action 장르 선호도
 - factor 2: Romance 장르 선호도
- $P(u, k)$ 는 사용자 u 의 k 번째 장르 선호도
- 예시:
 - $P(1, 1) = 0.94$ (사용자 1의 Action 선호도)
 - $P(1, 2) = 0.96$ (사용자 1의 Romance 선호도)

아이템-잠재요인 행렬 $Q(i, k)$

- 아이템 i 가 각 장르(잠재요인 k)를 얼마나 반영하고 있는지를 나타내는 값
- factor 1: 아이템의 Action 성분
- factor 2: 아이템의 Romance 성분
- $Q(i, k)$ 는 아이템 i 의 k 번째 장르 관련 요소
- $Q.T(k, i)$ 로 전치(transpose)하여 계산 시 사용
 - $Q(i, k) = Q.T(k, i)$

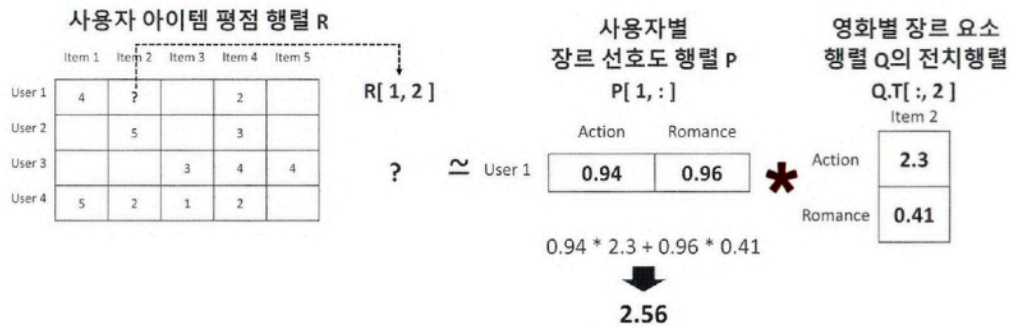


사용자-아이템 평점의 의미

- 사용자가 영화에 준 평점은 단순한 숫자가 아니라, 사용자의 영화 장르에 대한 선호도와 해당 영화의 장르적 특성의 내적 상호작용 결과로 이해할 수 있음.
- 예:
 - 사용자가 액션 영화를 좋아하고, 영화 자체가 액션 성향이 크다면 → 높은 평점 부여 가능성이 높음.

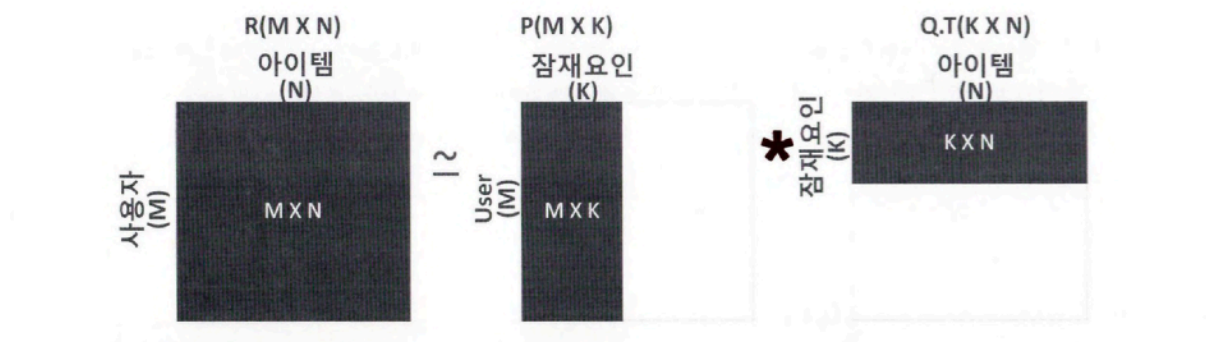
예측 평점 계산 원리

- 예측 평점은 사용자의 장르별 선호도 벡터와 영화의 장르별 특성 벡터의 내적(dot product) 결과로 계산됨.



b. 행렬 분해의 이해

- 다차원의 매트릭스를 저차원 매트릭스로 분해하는 기법, SVD, NFM
- 분해: 하나의 복잡한 행렬을 두 개 이상의 단순한 행렬의 곱으로 분해하는 것
- M개의 유저 행, N개의 아이템 열을 가진 평점 행렬 R은 M X N 차원으로 구성, 행렬 분해를 통해 사용자 - K 차원 잠재 요인 행렬 P + K 차원 잠재 요인 - 아이템 행렬 Q, $R \approx PQ^T$



- 행렬 내에서 널 값을 많이 가지는 고차원의 희소 행렬 R은 저차원 밀집 행렬인 P와 Q행렬로 분해 가능
- 평점 데이터: $r_{u,i} = p_u \cdot q_i^T$
- p는 u행 사용자의 벡터, q는 Q 행렬의 i행 아이템 벡터의 전치 벡터
- 사용자 아이템 평점 행렬의 미정 값을 포함한 모든 평점 값은 행렬 분해를 통해 얻어진 P 행렬과 Q.T행렬의 내적을 통해 예측 평점으로 다시 계산 가능
- SVD 방식 활용 → 널 값이 없는 행렬에만 적용 가능, 확률적 경사 하강법/ALS 방식을 이용해 SVD 수행

C. 확률적 경사 하강법을 이용한 행렬 분해

→ P와 Q행렬로 계산된 예측 R 행렬 값이 실제 R 행렬 값과 가장 최소의 오류를 가질 수 있도록 반복적인 비용 함수 최적화

- 전반적인 절차

1. P와 Q를 임의의 값을 가진 행렬로 설정합니다.
2. P와 Q.T 값을 곱해 예측 R 행렬을 계산하고 예측 R 행렬과 실제 R 행렬에 해당하는 오류 값을 계산합니다.
3. 이 오류 값을 최소화할 수 있도록 P와 Q 행렬을 적절한 값으로 각각 업데이트합니다.
4. 만족할 만한 오류 값을 가질 때까지 2, 3번 작업을 반복하면서 P와 Q 값을 업데이트해 근사화합니다.

- L2 규제를 고려한 비용 함수식:
$$\min \sum (r_{u,i} - p_u q_i)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$
- 비용 함수를 최소화하기 위해 새롭게 업데이트되는 p와 q 계산

$$\begin{aligned} p'_u &= p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u) \\ q'_i &= q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i) \end{aligned}$$

- 평점 행렬을 경사 하강법을 이용해 행렬 분해: L2 규제 반영, 실제 R 행렬 값과 예측 R 행렬 값의 차이를 최소화하는 방향성, 업데이트 값 반복 수행 → 최적화된 예측 R 행렬 구하는 방식 = SGD 기반의 행렬 분해

```
import numpy as np

# 원본 행렬 R 생성, 분해 행렬 P와 Q 초기화, 잠재 요인 차원 K는 3으로 설정.
R = np.array([[4, np.NaN, np.NaN, 2, np.NaN],
              [np.NaN, 5, np.NaN, 3, 1],
              [np.NaN, np.NaN, 3, 4, 4],
              [5, 2, 1, 2, np.NaN]])

num_users, num_items = R.shape
K=3

# P와 Q 행렬의 크기를 지정하고 정규 분포를 가진 임의의 값으로 입력합니다.
np.random.seed(1)
P = np.random.normal(scale=1./K, size=(num_users, K))
Q = np.random.normal(scale=1./K, size=(num_items, K))
```

→ 원본 행렬 R을 다시 P와 Q.T의 내적으로 예측 행렬을 만드는 예제

→ 랜덤 값 초기화, 잠재 요인 차원 3 설정

```
from sklearn.metrics import mean_squared_error

def get_rmse(R, P, Q, non_zeros):
    error = 0
    # 두 개의 분해된 행렬 P와 Q.T의 내적으로 예측 R 행렬 생성
    full_pred_matrix = np.dot(P, Q.T)

    # 실제 R 행렬에서 널이 아닌 값의 위치 인덱스 추출해 실제 R 행렬과 예측 행렬의 RMSE 추출
    x_non_zero_ind = [non_zero[0] for non_zero in non_zeros]
    y_non_zero_ind = [non_zero[1] for non_zero in non_zeros]
    R_non_zeros = R[x_non_zero_ind, y_non_zero_ind]
    full_pred_matrix_non_zeros = full_pred_matrix[x_non_zero_ind, y_non_zero_ind]
    mse = mean_squared_error(R_non_zeros, full_pred_matrix_non_zeros)
    rmse = np.sqrt(mse)

    return rmse
```

→ 실제 R 행렬과 예측 행렬의 오차를 구하는 get_rmse() 함수 생성

→ 실제 R 행렬의 널이 아닌 행렬 값의 위치 인덱스 추출, 조합된 예측 행렬 값의 RMSE 값 반환

```

# R > 0 인 행 위치, 열 위치, 값을 non_zeros 리스트에 저장.
non_zeros = [(i, j, R[i,j]) for i in range(num_users) for j in range(num_items) if R[i,j] > 0]

steps=1000
learning_rate=0.01
r_lambda=0.01

# SGD 가법으로 P와 Q 매트릭스를 계속 업데이트.
for step in range(steps):
    for i, j, r in non_zeros:
        # 현재 값과 예측 값의 차이를 오차로 간주함
        eij = r - np.dot(P[i, :], Q[j, :].T)
        # Regularization을 반영한 SGD 업데이트 공식 적용
        P[i, :] = P[i, :] + learning_rate*(eij * Q[j, :].T) + r_lambda*P[i, :]
        Q[j, :] = Q[j, :] + learning_rate*(eij * P[i, :].T) + r_lambda*Q[j, :]

    rmse = get_rmse(R, P, Q, non_zeros)
    if (step % 50) == 0:
        print("### Iteration step : ", step, " rmse : ", rmse)

```

→ 널 값을 제외한 데이터의 행렬 인덱스 추출

→ steps: SGD를 반복 업데이트할 횟수

→ learning_rate: SGD의 학습률

→ r_lambda: L2 Regularization 계수

```

pred_matrix = np.dot(P, Q.T)
print('예측 행렬:\n', np.round(pred_matrix, 3))

```

→ P*Q.T로 예측 행렬을 만들어 출력

[Output]

```

예측 행렬:
[[3.991 0.897 1.306 2.002 1.663]
 [6.696 4.978 0.979 2.981 1.003]
 [6.677 0.391 2.987 3.977 3.986]
 [4.968 2.005 1.006 2.017 1.14]]

```

→ 원본 행렬과 널이 아닌 값은 큰 차이 안 남, 널인 값은 새로운 예측값으로 채워짐

08. 파이썬 추천 시스템 패키지 - Surprise

a. Surprise 패키지 소개

- 파이썬 기반에서 사이킷런과 유사한 API와 프레임워크 제공

B. Surprise를 이용한 추천 시스템 구축

- 관련 모듈 임포트
- 추천을 위한 데이터 세트 로딩
- Dataset 클래스 이용해서 데이터 로딩 가능...
- Surprise 패키지를 이용한 추천 데이터 로딩 순서

① Surprise에서는 Dataset 클래스를 통해서만 데이터 로딩 가능

② Surprise가 처리 가능한 데이터 포맷은 다음과 같음 (Row 레벨 형태)

데이터는 userId(사용자 ID), movielid(영화 ID), rating(평점) 형태여야 함

→ 즉, 한 줄(row)에 한 개의 user-item-rating 정보가 있어야 함
예시:

③ Surprise는 무비렌즈(MovieLens) 데이터셋을 공식 지원

load_builtin() API를 통해 무비렌즈 과거 버전 데이터를 불러올 수 있음
대표 데이터셋:

ml-100k (10만 개 평점)

ml-1m (100만 개 평점)

단, 이 데이터는 먼저 공식 웹사이트에서 직접 다운로드하여 로컬에 저장해야 함

④ 로드된 데이터는 Surprise의 train_test_split() API로 분리

train_test_split()을 사용해 학습용(train)과 테스트용(test) 데이터 세트로 나누어 사용 가능

- SVD로 잠재 요인 협업 필터링 수행
- train_test_split() 으로 분리된 학습 데이터 세트에 적용
- algo = SVD() 처럼 알고리즘 객체를 생성, fit 수행하여 학습 데이터 세트 기반으로 추천 알고리즘 학습
- 학습된 추천 알고리즘 기반 테스트 데이터 세트에 대해 추천 수행
- test() 메서드, 입력된 데이터 세트에 대해 추천 데이터 세트 생성
- predict() 메서드, 개별 사용자와 영화에 대한 추천 평점 반환

```
predictions = algo.test( testset )
print('prediction type :', type(predictions), ' size:', len(predictions))
print('prediction 결과의 최초 5개 추출')
predictions[:5]
```

[Output]

```
prediction type : <class 'list'> size: 25000prediction 결과의 최초 5개 추출(Prediction(uid='120',
iid='282', r_ui=4.0, est=3.5114147666251547, details={'was_impossible': False}), Prediction(uid='882',
iid='291', r_ui=4.0, est=3.573872419581491, details={'was_impossible': False}), Prediction(uid='535',
iid='567', r_ui=5.0, est=4.033583485472447, details={'was_impossible': False}), Prediction(uid='697',
iid='244', r_ui=5.0, est=3.8463639495936985, details={'was_impossible': False}), Prediction(uid='751',
iid='385', r_ui=4.0, est=3.1807542478219157, details={'was_impossible': False}))
```

→ test() 메서드 실행, 추천 영화 평점 데이터 생성 후 최초 5개 추출 예제

→ 호출 결과 파이썬 리스트, 입력 인자 데이터 세트의 크기와 같은 25,000개

→ 반환 리스트 객체: 25000개 Prediction 객체 보유

→ 영화 아이디, 실제 평점 정보 기반 추천 예측 평점 데이터를 튜플 형태로

```
[ (pred.uid, pred.iid, pred.est) for pred in predictions[:3] ]
```

[Output]

```
[('120', '282', 3.5114147666251547),
 ('882', '291', 3.573872419581491),
 ('535', '567', 4.033583485472447)]
```

→ 3개의 Prediction 객체에서 uid, iid, est 속성을 추출한 예제

```
# 사용자 아이디, 아이템 아이디는 문자열로 입력해야 함.
uid = str(196)
iid = str(302)
pred = algo.predict(uid, iid)
print(pred)
```

[Output]

```
user: 196 item: 302 r_ui = None est = 4.49 {'was_impossible': False}
```

→ predict() 이용 추천 예측

→ 개별 사용자의 아이템에 대한 추천 평점 예측

→ 개별 사용자 아이디, 아이템 아이디 입력 시 추천 예측 평점 포함 저음 반환

→ rmse를 이용해 평가 결과 확인: 0.9467

b. Surprise 주요 모듈 소개

- Dataset: uid, iid, rating 데이터가 로우 레벨로 된 데이터 세트만 적용 가능

- API: Dataset.load_builtin(name = 'ml-100k') / Dataset.load_from_file(file_path, reader) / Dataset.load_from_df(df, reader)
- OS 파일 데이터를 Surprise 데이터 세트로 호출
 - 맨 처음 위치에 칼럼명을 헤더로 가지고 있는 파일의 헤더 삭제
 - Reader 클래스 활용, 데이터 파일의 파싱 포맷 정의, 데이터 세트로 로드
 - 생성자에 각 필드의 칼럼명과 칼럼 분리 문자, 최소최대 평점 입력 후 객체 생성, Reader 객체 참조해서 데이터 파일을 파싱하며 로딩

```
from surprise import Reader

reader = Reader(line_format='user item rating timestamp', sep=',', rating_scale=(0.5, 5))
data=Dataset.load_from_file('./ml-latest-small/ratings_noh.csv', reader=reader)
```

- 무비렌드 데이터 형식
- SVD 행렬 분해 기법 이용 추천 예측
- 잠재 요인 크기 K값을 나타내는 파라미터 n_factors = 50
- RMSE 로 평가 ⇒ RMSE: 0.8682

C, 판다스 DF에서 Surprise 데이터 세트로 로딩

```
import pandas as pd
from surprise import Reader, Dataset

ratings = pd.read_csv('./ml-latest-small/ratings.csv')
reader = Reader(rating_scale=(0.5, 5.0))

# ratings DataFrame에서 칼럼은 사용자 아이디, 아이템 아이디, 평점 순서를 지켜야 합니다.
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)
trainset, testset = train_test_split(data, test_size=25, random_state=0)

algo = SVD(n_factors=50, random_state=0)
algo.fit(trainset)
predictions = algo.test(testset)
accuracy.rmse(predictions)
```

[Output]

RMSE: 0.8682

→ ratings 파일을 DF로 로딩한 ratings에서 Surprise 데이터 세트로 로딩, 파라미터 입력

→ SVD 추천 예측

d. Surprise 추천 알고리즘 클래스

클래스명	설명
SVD	행렬 분해를 통한 잠재 요인 협업 필터링을 위한 SVD 알고리즘.
KNNBasic	최근접 이웃 협업 필터링을 위한 KNN 알고리즘.
BaselineOnly	사용자 Bias와 아이템 Bias를 감안한 SGD 베이스라인 알고리즘.

파라미터명	내용
n_factors	잠재 요인 K의 개수. 디폴트는 100. 커질수록 정확도가 높아질 수 있으나 과적합 문제가 발생할 수 있습니다.
n_epochs	SGD(Stochastic Gradient Descent) 수행 시 반복 횟수. 디폴트는 20.
biased (bool)	베이스라인 사용자 편향 적용 여부이며 디폴트는 True입니다.

- 테스트 결과 (벤치마크: Core i5 7th gen(2.5 GHz), 8G RAM상에서 100k 데이터 세트)

알고리즘 유형	RMSE	MAE	Time
SVD	0.934	0.737	0:00:11
SVD++	0.92	0.722	0:09:03
NMF	0.963	0.756	0:00:15
Slope One	0.946	0.743	0:00:08
k-NN	0.98	0.774	0:00:10
Centered k-NN	0.951	0.749	0:00:10
k-NN Baseline	0.931	0.733	0:00:12
Co-Clustering	0.963	0.753	0:00:03
Baseline	0.944	0.748	0:00:01

→ 시간이 오래 걸리는 RMSE, MAE 성적은 가장 좋음

→ SVD, k-NN Baseline이 가장 성능 평가 수치 좋음

→ Baseline 결합한 경우 성능 평가 수치 대폭 향상

e. 베이스라인 평점

- 베이스라인 평점 = 전체 평균 평점 + 사용자 편향 점수 + 아이템 편향 점수

f. 교차 검증과 하이퍼 파라미터 튜닝

- cross_validate() 와 GridSearchCV 클래스 제공

```
from surprise.model_selection import cross_validate

# pandas DataFrame에서 Surprise 데이터 세트로 데이터 로딩
ratings = pd.read_csv('./ml-latest-small/ratings.csv') # reading data in pandas df
reader = Reader(rating_scale=(0.5, 5.0))
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)

algo = SVD(random_state=0)
cross_validate(algo, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

- Rating 을 DF로 로딩한 데이터를 5개의 학습검증 폴드 데이터 세트로 분리해 교차 검증 수행, RMSE, MAE로 성능 평가 진행

- 알고리즘 객체, 데이터, 성능 평가 방법, 폴드 데이터 세트 개수 입력

- 폴드별 성능 평가 수치와 전체 폴드의 평균 성능 평가 수치 함께 보여줌

[Output]

```
0.8769866363866617
({'n_epochs': 20, 'n_factors': 50})
```

- GridSearchCV도 교차 검증을 통한 하이퍼 파라미터 최적화 수행

f. Surprise를 이용한 개인화 영화 추천 시스템 구축

- DatasetAutoFolds 클래스 이용, 메서드 호출 시 전체 데이터 학습 데이터 세트화 가능
- SVD를 이용한 학습 수행
- Uid = 9 인 사용자 지정, mid = 42
- predict() 메서드 이용 추천 예상 평점 알아내기, 모두 문자열 값이어야 함
- 사용자가 평점을 매기지 않은 전체 영화 추출 후 예측 평점 순으로 영화 추천
 - 추천 대상이 되는 영화 추출, 새로운 get_unseen_surprise() 함수 생성, 아이디 9 사용자가 평점 안 매긴 영화 정보 반환
 - 추천 대상 영화 9696/9742, SVD 클래스 활용하여 높은 예측 평점을 가진 순으로 영화 추천
 - recomm_movie_by_surprise()
 - 추천 대상 영화 모두 대상으로 추천 알고리즘 객체 predict() 메서드 호출, 결과 객체를 리스트 객체로 저장

[Output]

```
평점 매긴 영화수: 46 추천대상 영화수: 9696 전체 영화수: 9742
#### Top-10 추천 영화 리스트 ####
Usual Suspects, The (1995) : 4.386302135708814
Star Wars: Episode IV - A New Hope (1977) : 4.281663842987387
Pulp Fiction (1994) : 4.278152632122759
Silence of the Lambs, The (1991) : 4.226073566468876
Godfather, The (1972) : 4.1918097904381995
Streetcar Named Desire, A (1951) : 4.154746591122658
Star Wars: Episode V - The Empire Strikes Back (1980) : 4.122016128534504
Star Wars: Episode VI - Return of the Jedi (1983) : 4.108009609093436
Goodfellas (1990) : 4.083464936588478
Glory (1989) : 4.07887165526957
```

→ 알고리즘 객체 메서드를 평점이 없는 영화에 반복 수행, 결과 list 객체로 저장

→ sortket_est 함수 정의하여 est 값으로 정렬

반환값의 내림차순으로 정렬 수행, top_n개 최상위 값 추출

영화 아이디, 추천 예상 평점, 제목 추출